

**M.Abdullah.Mahmood**

**BCS-3B**

**22f3289**

**DS lab – 4**

**To: Sir Abdul Qadeer**

---

**Task 2 :**

**Code:**

```
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node* next;
    Node()
    {
        data = 0;
        next = nullptr;
    }

    Node(int d) : data(d), next(nullptr) {};
};
class Stack
{
private:
    Node* top; // Pointer to the top node
public:
    // Constructor
    Stack()
    {
        top = NULL;
    }

    // Destructor to free memory
    ~Stack()
```

```

{
    Node* temp = top;
    while (top != NULL) {
        delete temp;
        temp = temp->next;
    }
}

// Check if the stack is empty
bool isEmpty()
{
    return (top == 0);
}

// Push an element onto the stack
void push(int value)
{
    Node* temp = new Node(value);
    temp->next = top;
    top = temp;
}

// Pop the top element from the stack
void pop()
{
    if (isEmpty()) {
        cout << "stack is empty \n";
        return;
    }
    Node* temp = new Node();
    temp = top;

    top = top->next;
    delete temp;
    return;
}

// Return the top element without removing it
int peekAtTop()
{
    Node* temp = top;
    if (isEmpty()) {
        cout << "stack is empty \n";
        return 0;
    }
    return (temp->data);
}

// Display the elements in the stack
void display()
{
    Node* temp = top;
    if (isEmpty()) {
        cout << "stack is empty \n";
        return;
    }

    while (temp) {
        cout << temp->data;

```

```

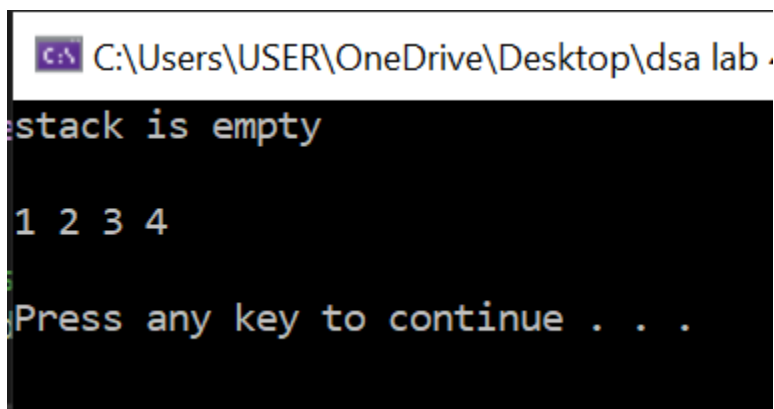
        temp = temp->next;
    }
    return;
}
};
int main()
{
    Stack stack;
    if (stack.isEmpty()) {
        cout << "stack is empty \n";
    }
    stack.push(4);
    stack.push(3);
    stack.push(2);
    stack.push(1);
    stack.push(0);

    stack.pop();
    stack.peekAtTop();
    cout << endl;
    stack.display();
    cout << endl;

    system("pause");
    return 0;
}

```

## Output



```

C:\Users\USER\OneDrive\Desktop\dsa lab .
stack is empty

1 2 3 4

Press any key to continue . . .

```

# Task 1 :

```
#include<iostream>
using namespace std;

class medicalRecord {
public:
    int recordId, dateOfCon;
    string docName, diagnosis, persc;
    medicalRecord* next, * prev;

    medicalRecord(int recld, int date, string doc, string diag, string pers) {
        recordId = recld;
        dateOfCon = date;
        docName = doc;
        diagnosis = diag;
        persc = pers;
        next = NULL;
        prev = NULL;
    }

    medicalRecord():next(nullptr),prev(nullptr) {};
    ~medicalRecord() {};
};

class patientRecord {
public:
    int UniqueId;
    string name;
    medicalRecord* medical_record;
    patientRecord* next, * prev;

    patientRecord() {};
    patientRecord(int Uid, string nam) {
        UniqueId = Uid;
        name = nam;
        next = NULL;
        prev = NULL;
        medical_record = NULL;
    }
    ~patientRecord() {};
};

class HIC {
public:
    patientRecord* Node = new patientRecord();

    bool isEmpty() {
        return Node == NULL;
    }

    void insertPatient(string n) {
        if (isEmpty()) {
            cout << "enter ID : ";
            cin >> Node->UniqueId;
            cout << "enter patient name : ";
```

```

    cin >> Node->name;
    cout << "enter doc name : ";
    cin>>Node->medical_record->docName;
    cout << "enter record ID : ";
    cin >> Node->medical_record->recordId;
    cout << "enter Date of consultation : ";
    cin >> Node->medical_record->dateOfCon;
    cout << "enter diagnosis : ";
    cin >> Node->medical_record->diagnosis;
    cout << "enter perscription : ";
    cin >> Node->medical_record->persc;
}
else {
    patientRecord* temp = Node;
    while (temp->next != NULL) {
        if (temp->name == n) {
            medicalRecord* newNode = new medicalRecord();
            cout << "enter record id : \n";
            cin >> newNode->recordId;
            cout << "enter date of consultation : \n";
            cin >> newNode->dateOfCon;
            cout << "enter diagnosis : \n";
            cin >> newNode->diagnosis;
            cout << "enter doc name : \n";
            cin >> newNode->docName;
            cout << "enter perscription : ";
            cin >> newNode->persc;
            newNode->next = NULL;

            temp->medical_record->next = newNode;
            newNode->prev = temp->medical_record;
        }
        temp = temp->next;
    }
    patientRecord* newNode = new patientRecord();

    cout << "enter ID : ";
    cin >> newNode->Uniqueld;
    cout << "enter patient name : ";
    cin >> newNode->name;

    cout << "enter doc name : ";
    cin >> newNode->medical_record->docName;
    cout << "enter record ID : ";
    cin >> newNode->medical_record->recordId;
    cout << "enter Date of consultation : ";
    cin >> newNode->medical_record->dateOfCon;
    cout << "enter diagnosis : ";
    cin >> newNode->medical_record->diagnosis;
    cout << "enter perscription : ";
    cin >> newNode->medical_record->persc;

    temp->next = newNode;
    newNode->prev = temp;
}
}

void SearchMedicalRecord(string name, int id) {
    patientRecord* temp = new patientRecord();

```

```

while (temp != NULL) {
    if (temp->name == name) {
        while (temp->medical_record->next != NULL) {
            if (temp->medical_record->recordId == id) {
                cout << temp->medical_record->dateOfCon << " ";
                cout << temp->medical_record->diagnosis << " ";
                cout << temp->medical_record->docName << " ";
                cout << temp->medical_record->recordId << "\n";
            }
            temp->medical_record = temp->medical_record->next;
        }
        temp = temp->next;
    }
}

}
~HIC() {}

};

int main()
{
    HIC p1;
    p1.insertPatient("abdullah");

    return 0;
}

```

Output;

**UNCOMPLETE**